1. **What is Azure Service Bus**

   Ans - Azure Service Bus is a fully managed message broker service from Microsoft Azure that enables applications, services, and systems to communicate with each other asynchronously.

2. **What are core concept of Azure Service Bus**

   Ans - Azure Service Bus works like a smart post office where one application sends messages to a queue or topic, the Service Bus safely stores and manages those messages with features like retries, ordering, dead-lettering, and transactions, and another application later receives and processes them without both systems needing to communicate directly at the same time, which helps build scalable, reliable, and loosely coupled distributed applications.

3. **What is Queue, Topics and Subscription?**

   Ans - **A Queue** works like a single delivery box where one sender sends a message and only one receiver processes it .

   **A Topic** works like a broadcasting station where one sender publishes a message and multiple receivers can listen to it through Subscriptions.

   **A Subscription** behaves like its own virtual queue so different services such as billing, inventory, and notification can independently receive and process the same message according to their own business needs.

4. **What is difference between Queue and Topics. When to use both of them?**

   Ans - A Queue in Azure Service Bus is used when a message should be processed by only one consumer, while a Topic is used when the same message needs to be shared with multiple consumers independently.

   For example, if an ATM machine sends a “Cash Withdrawal Request” and only one banking service should process that transaction, then Queue is the correct choice because duplicate processing could create problems, but if an e-commerce application creates an order and multiple systems like inventory, billing, notification, analytics, and shipping all need the same order information, then Topic is the better choice because one published message can be distributed to multiple subscriptions.

   You should use a Queue when you want task distribution, background processing, load balancing, or worker-based processing where only one receiver should handle the message, and you should use a Topic when you are building event-driven or microservice architectures where many services must react to the same business event independently without tightly coupling systems together.

5. **What are partitions in service bus and when should they be used?**

   Ans - Partitions in Azure Service Bus are used to split messages across multiple internal message brokers and storage units so the load is distributed automatically, which improves scalability, availability, and throughput when a large number of messages are being sent or received at the same time.

   Without partitioning, all messages of a queue or topic are stored in a single message broker, so very high traffic can become a bottleneck, but with partitioning enabled, Azure Service Bus spreads messages across multiple partitions internally while still appearing as a single queue or topic to the application.

   For example, imagine a food delivery application during a festival sale where millions of users place orders simultaneously; if all order messages go to one single queue storage, processing may slow down, but with partitioning enabled, Azure Service Bus distributes those messages across multiple partitions so many consumers can process them efficiently in parallel.

   You should use partitioning when your application expects high message volume, high throughput, large-scale microservices communication, or unpredictable traffic spikes, especially in enterprise systems

like banking, e-commerce, logistics, or IoT platforms, but for small applications or when strict message ordering is extremely important, partitioning may not be the best choice because messages with different partition keys can be processed independently across partitions.

Partitioning is not enabled by default in Azure Service Bus, so while creating the Queue or Topic you must explicitly enable it.

When you create a Queue or Topic in the Azure Portal, there is an option called Enable partitioning, and you simply check that checkbox before creating the resource. After creation, Azure internally creates multiple partitions automatically and distributes messages across them.

6. **How does service bus ensure high availability?**
   Ans - Azure Service Bus ensures high availability by automatically replicating messages and distributing workloads across multiple servers and infrastructure inside Azure so that if one machine, broker, or storage node fails, another instance can continue processing messages without application downtime.

   When a sender pushes a message into Service Bus, the message is not stored in just one physical server; Azure keeps redundant copies internally and continuously monitors the health of the messaging infrastructure, so hardware failures, network failures, or server crashes are handled automatically by the platform.

   If partitioning is enabled, messages are spread across multiple partitions hosted on different brokers, which further improves fault tolerance and scalability because traffic is not dependent on a single machine.

   Azure Service Bus also supports features like geo-disaster recovery, retry policies, dead-letter queues, duplicate detection, and transactional processing, which help applications continue operating even during temporary failures or processing issues.

7. **What is geo-disaster and what is the role of regions and geo-disaster recovery?**
   Ans - A Region in Azure is a physical geographic location containing multiple datacenters, such as Central India, East US, or West Europe, where your Azure resources like Azure Service Bus are hosted.

   A geo-disaster means a large-scale failure affecting an entire region, such as earthquakes, floods, power failures, war, major network outages, or complete datacenter failures, where the whole region may become unavailable instead of just a single server failing.

   Geo-disaster recovery in Azure Service Bus is designed to protect applications from such regional failures by pairing a primary namespace in one region with a secondary namespace in another region, so if the primary region goes down, applications can fail over to the secondary region and continue operating with minimal downtime.

8. What messaging patterns are supported by Azure Service Bus?
   Ans - Azure Service Bus supports multiple messaging patterns that help distributed applications communicate reliably and asynchronously depending on business requirements.

   The most common pattern is **Point-to-Point messaging**, which uses Queues where one sender sends a message and only one receiver processes it, such as an order processing system where each order should be handled only once by one worker service.

Another important pattern is **Publish/Subscribe**, which uses Topics and Subscriptions where one sender publishes a message and multiple subscribers receive independent copies, such as an e-commerce application publishing an “Order Created” event that is consumed separately by inventory, billing, shipping, and notification services.

Azure Service Bus also supports the **Request-Reply pattern**, where one application sends a request message and waits for a response from another service using correlation IDs and reply queues, similar to a payment gateway validating a transaction and returning approval status.

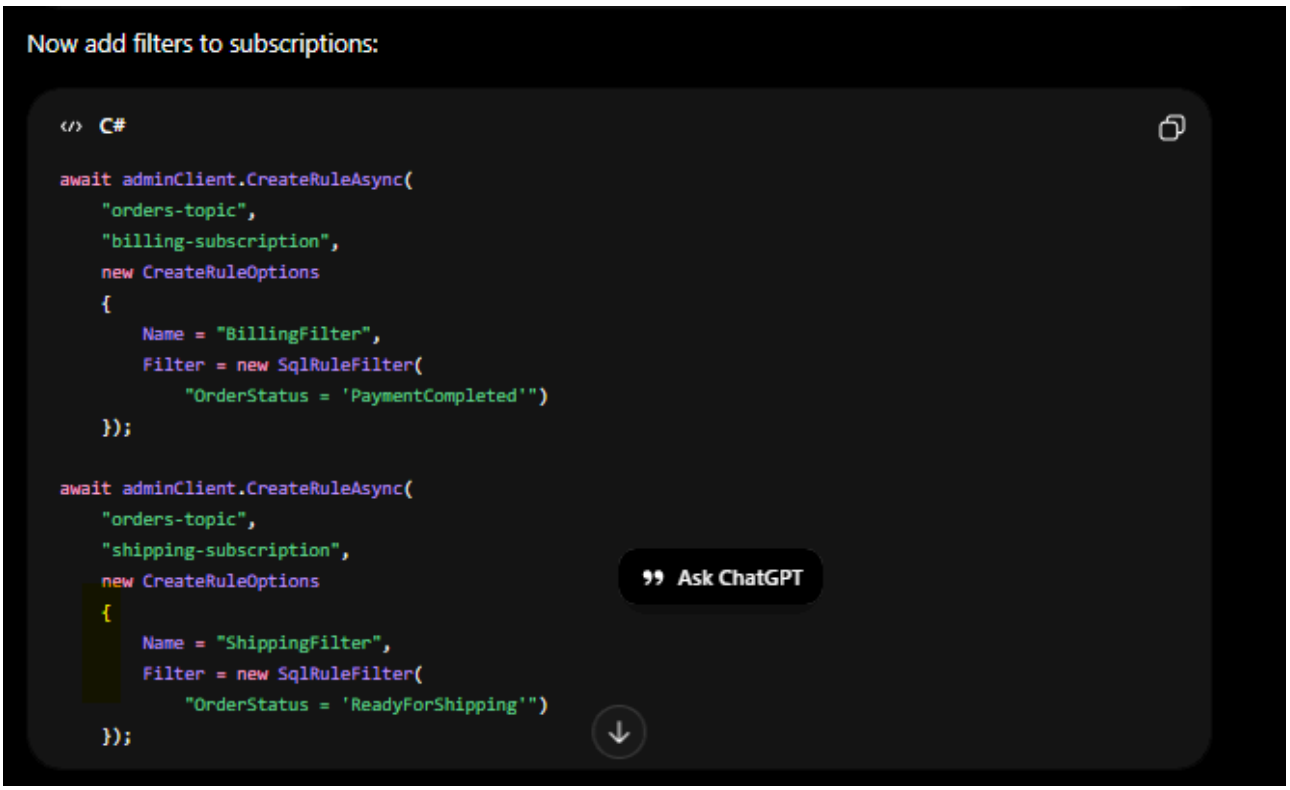
Another supported pattern is **Competing Consumers**, where multiple receivers listen to the same queue and Azure Service Bus distributes messages among them for load balancing and parallel processing, which is commonly used in high-volume systems like banking or ticket booking platforms.

It also supports **Scheduled Messaging**, where messages can be delivered at a future time, such as sending reminder notifications or delayed order processing.

Additionally, Azure Service Bus supports **Session-based messaging**, which guarantees ordered processing for related messages by grouping them with a session ID, often used in workflows like financial transactions or conversational chat systems where sequence matters.

9. How can you implement message routing using topics and filters

Ans - Filters ensure each service receives only relevant business events, reducing unnecessary processing and improving scalability.




10. What is PeekLock and how does it work internally?

Ans - **Peek-Lock** is a message retrieval mode in Azure Service Bus that ensures a message is processed exactly once without data loss. It temporarily locks a pulled message so no other consumers can see it, and waits for your application to explicitly confirm successful processing before deleting it.

**Peek & Lock:** A receiver pulls the message. Azure Service Bus isolates it and places a lock for a predefined duration (defaulting to 1 minute, configurable up to 5 minutes).

**Processing**: The application attempts to process the payload.
**Completion** (Success): If successful, the app sends a Complete() command, permanently removing the message from the queue.
**Abandonment** (Failure): If the app crashes or encounters an error, it calls Abandon(). The lock drops immediately, making the message available again for other consumers.
**Key Operations—-----**
**Complete** (CompleteAsync): Deletes the message from the queue after successful processing.
**Abandon** (AbandonAsync): Immediately releases the lock and increments the message's delivery count, making it available to be processed again.
**Dead-Lette**r (DeadLetterAsync): Moves problematic messages to a sub-queue for manual inspection so they don't block the main flow.
**Renew Lock** (RenewLockAsync): Resets the lock timer if your application needs more time to process the message.

11. What is the impact of abandoning a message?
Ans - Abandoning a message in Azure Service Bus means the consumer tells Service Bus that it could not process the message successfully right now, so the message lock is released immediately and the message becomes available again for reprocessing by the same or another consumer. When message is abandoned, DeliveryCount is happening, maximum 10 time a message can abandon after that move to DLQ.

```
AbandonMessageAsync()
```

12. What is defer in azure service bus?
Ans - Do not process this message now, but keep it safely so I can process it later. Automatically create SequanceNo and assign to the message. Same sequence number we need to keep in db or other place once we need we can process the message by sequanceNo

**13. DLQ how we handle too many message?**
Ans - If too many messages are going to DLQ, first we analyze the root cause using dead-letter reason and error description. Then we create a DLQ processor service or scheduled job that reads DLQ messages in batches, categorizes temporary vs permanent failures, retries only valid retryable messages, logs errors, sends alerts, and avoids infinite replay loops using retry limits.
Step 1 — Identify Root Cause, Step 2 — Categorize Errors, Step 3 — Create DLQ Processor, Step 4 — Read DLQ Messages in Batch, Step 5 — Log and Store Errors, Step 6 — Replay Only Valid Messages, Step 7 — Avoid Infinite Retry Loop, Step 8 — Monitoring & Alerts

14.